

Day 11 - Interview Questions

1. What is inheritance in Python, and why is it used?

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a new class (subclass or derived class) to inherit properties and methods from an existing class (superclass or base class). It is used to promote code reusability and establish relationships between classes.

2. How do you create a subclass in Python?

To create a subclass in Python, you define a new class and specify the superclass in parentheses after the class name.

3. What is a superclass and a subclass?

A superclass (or base class) is the class from which other classes inherit properties and methods. A subclass (or derived class) is a class that inherits from a superclass.

4. How do you access methods and attributes of a superclass from a subclass?

You can access superclass methods and attributes using the `super()` function.

5. Explain method overriding in Python.

Method overriding occurs when a subclass provides its implementation for a method that is already defined in the superclass. The overridden method in the subclass has the same name and parameters as the one in the superclass.

6. What is multiple inheritance in Python?

Multiple inheritance is a feature in Python that allows a class to inherit from more than one superclass. This means a subclass can inherit attributes and methods from multiple parent classes.

7. What is the `isinstance()` function used for in Python?

The `isinstance()` function is used to check if an object is an instance of a particular class or its subclasses. It returns `True` if the object is an instance of the specified class or a subclass, and `False` otherwise.

8. What is hybrid inheritance in Python?

Hybrid inheritance is a combination of different types of inheritance, such as single, multiple, multilevel, or hierarchical inheritance. It can create complex class hierarchies.

9. Are there any limitations or issues associated with multiple inheritance in Python?

Yes, multiple inheritance can lead to problems like the "diamond problem," where two classes that share a common base class are inherited by a single subclass. Python resolves this using the C3 linearization algorithm to determine method resolution order (MRO).

10. What is an exception in Python? Explain the purpose of the try, except, and finally blocks in exception handling.

An exception is an event that occurs during the execution of a program and disrupts the normal flow of the program.

The try block contains code that might raise an exception.

The except block handles exceptions that are raised in the try block.

The finally block contains code that will be executed regardless of whether an exception is raised or not.

11. What is the purpose of the finally block when handling exceptions, even if you don't intend to perform any cleanup?

The finally block is useful for ensuring that certain code, like closing files or releasing resources, is executed regardless of whether an exception occurs or not.

12. What is the assert statement in Python, and how is it used for exception handling?

The assert statement is used for debugging and testing. It raises an AssertionError if a given condition is False. It's often used to catch logical errors in code.

13. What is the difference between the except block with and without an exception type?

An except block without an exception type catches all exceptions. An except block with an exception type catches only exceptions of that specific type or its subclasses.

14. How can you raise a custom exception in Python?

You can raise a custom exception by using the raise statement followed by the exception type and an optional error message.

15. How can you access the exception message in an except block?

You can access the exception message by using the as keyword followed by a variable name in the except block, like except ExceptionType as e, where e is the variable that contains the exception instance.